

UNIVERSITÀ POLITECNICA DELLE MARCHE

INGEGNERIA INFORMATICA E
DELL'AUTOMAZIONE

Corso di
DATA SCIENCE

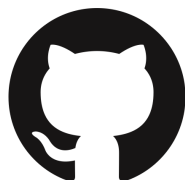


Analisi del Sentiment e della Brand Reputation su Twitter: Il caso Dell



Docenti:

Domenico Ursino
Christopher Buratti



Studenti:

Matteo Copertari
Kevin Giusti
Matteo Stronati
Jacopo Tarulli

Anno accademico 2025-2026

Indice

1	Introduzione	4
1.1	Dataset scelto	4
1.1.1	Caratteristiche del Dataset	5
1.2	Motivazioni della scelta	6
2	Analisi esplorativa dei dati (EDA)	7
2.1	Analisi quantitativa del Sentiment	7
2.2	Analisi delle emozioni	8
2.3	Analisi dei termini con WordCloud	9
2.3.1	Metodologia di implementazione	9
2.3.2	Visualizzazione dei risultati	9
3	Preprocessing ed embedding del testo	12
3.1	Strategia di suddivisione del dataset	12
3.2	Pipeline di preprocessing	12
3.3	Analisi Post-Processing	13
3.4	Rappresentazione vettoriale: Word2Vec	14
3.4.1	Addestramento del modello	14
3.4.2	Utilizzo nei modelli	15
4	Modelli testati	16
4.1	Modello 1: Naive Bayes	16
4.1.1	Configurazione	16
4.1.2	Risultati	16
4.2	Modello 2: Logistic Regression	17
4.2.1	Configurazione	17
4.2.2	Risultati	17
4.3	Modello 3: Support Vector Machine (SVM)	18
4.3.1	Configurazione	18
4.3.2	Risultati	19
4.4	Modello 4: Long Short-Term Memory (LSTM)	19
4.4.1	Configurazione	20
4.4.2	Risultati	20
4.5	Modello 5: Bidirectional Long Short-Term Memory (Bi-LSTM)	21
4.5.1	Configurazione	21
4.5.2	Risultati	22
4.6	Analisi finale e conclusione	23

5	Information Extraction	24
5.1	<i>Key-Phrase Extraction (KPE)</i>	24
5.1.1	Pre-processing	24
5.1.2	<i>KPE</i> mediante <i>spaCy</i> e <i>textacy</i>	26
5.1.3	Pipeline di <i>KPE</i> : confronto tra modelli <i>spaCy</i>	27
5.1.4	Risultati sperimentali <i>KPE</i>	30
5.2	<i>Named Entity Recognition (NER)</i>	36
5.2.1	Pre-processing	36
5.2.2	<i>NER</i> mediante <i>spaCy</i>	37
5.2.3	Pipeline <i>NER</i>	39
5.2.4	Risultati sperimentali <i>NER</i>	39

Elenco delle figure

1.1	Prime righe del dataset: si notano la struttura del testo e le etichette di sentiment ed emozione con i relativi score.	6
2.1	Distribuzione percentuale dei sentiment	7
2.2	Distribuzione delle 8 emozioni principali nel dataset	8
2.3	Distribuzione delle emozioni segmentata per sentiment	8
2.4	WordCloud generale dei termini	10
2.5	WordCloud separate per sentiment	11
3.1	Esempio di tokenizzazione di un tweet	14
4.1	Matrice di confusione del Gaussian Naive Bayes	17
4.2	Matrice di confusione della Logistic Regression	18
4.3	Matrice di confusione del Support Vector Machine	19
4.4	Andamento delle curve di loss e accuracy per il modello LSTM	20
4.5	Matrice di confusione del modello LSTM	21
4.6	Andamento delle curve di loss e accuracy per il modello Bi-LSTM	22
4.7	Matrice di confusione del modello Bi-LSTM	23
5.1	Keyphrase più frequenti	35
5.2	Relazione tra keyphrase e proprietà dei tweet	35
5.3	Keyphrase overlap	36
5.4	Analisi della distribuzione delle NER labels	44
5.5	Analisi della distribuzione delle NER labels con filtro sulla keyword Dell	45

1 Introduzione

Negli ultimi anni, l'analisi del linguaggio naturale (NLP) ha assunto un ruolo centrale nello studio dei dati provenienti dai social media. Piattaforme come Twitter (ora X) generano quotidianamente milioni di contenuti testuali non strutturati che rappresentano una miniera d'oro per le aziende che desiderano comprendere la percezione del proprio marchio.

L'obiettivo di questo progetto è applicare diverse tecniche di NLP per analizzare la *Brand Reputation* dell'azienda **Dell**. Nello specifico, il lavoro si propone di classificare il sentimento (**Sentiment Analysis**) dei tweet inviati dagli utenti, identificando se l'opinione espressa è positiva, negativa o neutra, e di esplorare le emozioni sottostanti (come rabbia, gioia o anticipazione). Inoltre, dai tweet è stata condotta un'**Information Extraction** avanzata, comprendente **Keyphrase Extraction** per identificare i termini chiave più rilevanti e **Named Entity Recognition (NER)** per estrarre entità nominate come prodotti, aziende e persone, fornendo così un'analisi contestualizzata e granulare dei contenuti discussi.

Il progetto segue una pipeline completa di Data Science che comprende:

- **Analisi Esplorativa (EDA)**: Per comprendere la distribuzione dei dati e i trend principali.
- **Pre-processing**: Per pulire e normalizzare il testo grezzo proveniente dai social.
- **Sentiment Analysis**: Confronto di modelli di Machine Learning e basati su reti neurali per la classificazione dei testi.
- **Information Extraction**: Sviluppo di algoritmi per i task di Keyphrase Extraction e NER.

1.1 Dataset scelto

Per questo studio è stato utilizzato il dataset **Sentiment & Emotions Labelled Tweets**, disponibile sulla piattaforma Kaggle¹. Si tratta di una raccolta di dati testuali focalizzata sulle interazioni degli utenti relative ai prodotti e al supporto tecnico Dell.

¹<https://www.kaggle.com/datasets/ankitkumar2635/sentiment-and-emotions-of-tweets>

1.1.1 Caratteristiche del Dataset

Il dataset è composto esattamente da **24.970 righe** e **8 colonne**. Ogni record rappresenta un singolo tweet arricchito da metadati e da etichette di classificazione pre-calcolate.

Nella Tabella 1.1 sono descritte nel dettaglio tutte le feature disponibili.

Nome Colonna	Descrizione
Datetime	Data e ora di pubblicazione del tweet (formato timestamp). Utile per eventuali analisi di trend temporale.
Tweet Id	Identificativo numerico univoco del tweet (<i>int64</i>).
Text	Il contenuto testuale grezzo del tweet. Rappresenta l'input principale per i modelli di NLP.
Username	L'handle dell'utente che ha postato il tweet. Utile in fase di pre-processing per anonimizzare i dati.
sentiment	La variabile target principale. Etichetta categorica che assume tre valori: <i>positive</i> , <i>negative</i> , <i>neutral</i> .
sentiment_score	Valore numerico (<i>float</i>) che indica la confidenza o probabilità associata all'etichetta di sentiment assegnata.
emotion	Etichetta categorica che descrive l'emozione predominante (es. <i>anger</i> , <i>joy</i> , <i>anticipation</i>).
emotion_score	Valore numerico (<i>float</i>) che indica la confidenza associata all'emozione rilevata.

Tabella 1.1: *Descrizione completa delle feature del dataset Dell.*

Una prima ispezione dei dati mostra come il testo contenga numerosi elementi di "rumore" tipici del linguaggio social, quali menzioni a utenti (es. @DellCares), hashtag, emoji, URL e punteggiatura non standard.

Di seguito, in Figura 1.1, è riportato un estratto delle prime righe del dataset grezzo per visualizzare la struttura tabellare.

Prime 5 righe:

Out[2]:	Datetime	Tweet Id	Text	Username	sentiment	sentiment_score	emotion	emotion_score
0	2022-09-30 23:29:15+00:00	1575991191170342912	@Logitech @apple @Google @Microsoft @Dell @Len...	ManjuSreedaran	neutral	0.853283	anticipation	0.587121
1	2022-09-30 21:46:35+00:00	1575965354425131008	@MK_habit_addict @official_stier @MortalKombat...	MiKeMcDnet	neutral	0.519470	joy	0.886913
2	2022-09-30 21:18:02+00:00	1575958171423752203	As @CRN celebrates its 40th anniversary, Bob F...	jfollett	positive	0.763791	joy	0.960347
3	2022-09-30 20:05:24+00:00	1575939891485032450	@dell your customer service is horrible especi...	daveccarr	negative	0.954023	anger	0.983203
4	2022-09-30 20:03:17+00:00	1575939359160750080	@zacokalo @Dell @DellCares @Dell give the man ...	heycamella	neutral	0.529170	anger	0.776124

Figura 1.1: Prime righe del dataset: si notano la struttura del testo e le etichette di sentiment ed emozione con i relativi score.

Dalle analisi preliminari emerge che il dataset copre diverse tipologie di interazioni, spaziando da **richieste di assistenza tecnica** (spesso associate a sentiment negativo) a **feedback su nuovi prodotti** (spesso associati a sentiment positivo o neutro).

1.2 Motivazioni della scelta

La scelta di analizzare i tweet relativi a Dell è motivata dalla natura "tecnica" e specifica del linguaggio utilizzato. A differenza di dataset generici, i commenti su prodotti tecnologici contengono spesso termini specifici che mettono alla prova la capacità di generalizzazione degli algoritmi di Embedding come **Word2Vec**. Inoltre, il dataset presenta le sfide tipiche descritte nella letteratura NLP per i social media:

- **Rumore:** Presenza di slang, errori di battitura ed emoji.
- **Contesto:** Necessità di comprendere il sarcasmo o le negazioni (es. "Not bad" vs "Bad").
- **Multiclasse:** La presenza della classe "neutra" aggiunge complessità rispetto alla semplice classificazione binaria.

Nel capitolo successivo, verranno esplorate nel dettaglio le statistiche descrittive del dataset attraverso tecniche di visualizzazione dati.

2 Analisi esplorativa dei dati (EDA)

L'analisi è stata condotta su un corpus di **24.970 tweet** riguardanti il brand Dell. Il dataset fornisce una doppia etichettatura: una basata sulla polarità (*sentiment*) e una basata sullo stato emotivo dell'utente (*emotion*), permettendo uno studio multidimensionale della reputazione online del brand.

2.1 Analisi quantitativa del Sentiment

Il primo passo dell'analisi esplorativa ha riguardato la distribuzione della variabile target. Come evidenziato nella Figura 2.1, il dataset presenta una distribuzione sbilanciata che riflette l'uso tipico dei social media nel settore tech.

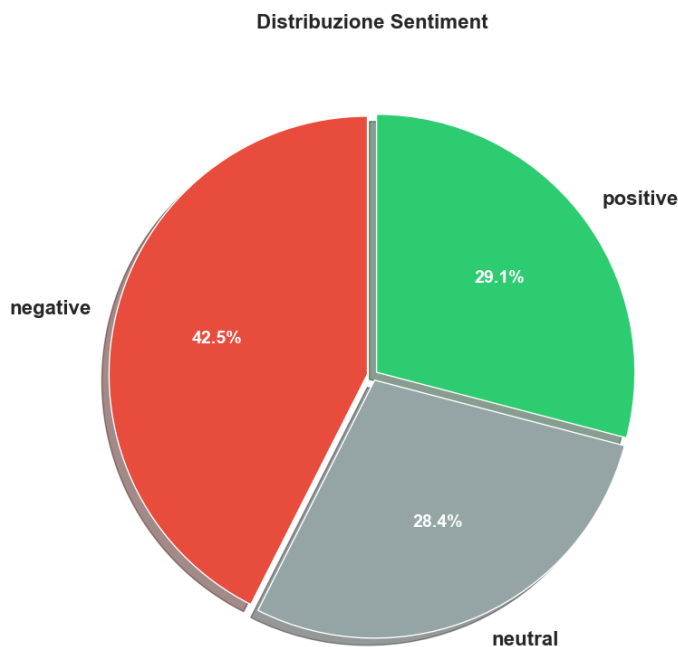


Figura 2.1: *Distribuzione percentuale dei sentiment*

La prevalenza della classe **negative** ($\sim 42\%$) conferma che Twitter è utilizzato prevalentemente come canale di sfogo o di richiesta di assistenza per problemi tecnici. Le classi *positive* e *neutral* si attestano su valori simili, coprendo il restante 58% del dataset.

2.2 Analisi delle emozioni

L'estensione dell'analisi alle otto emozioni principali (Figura 2.2) permette di mappare psicologicamente il contenuto dei tweet.

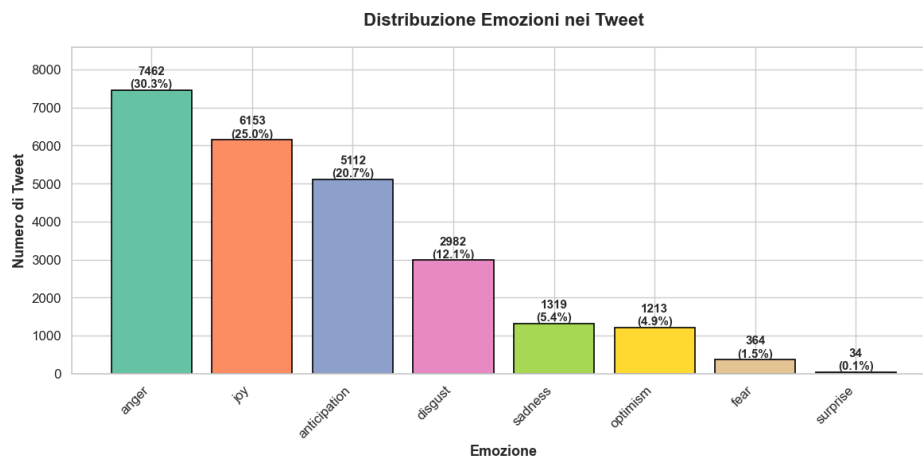


Figura 2.2: *Distribuzione delle 8 emozioni principali nel dataset*

Per approfondire la relazione tra polarità e stato d'animo, è stata analizzata la **distribuzione delle emozioni per ogni sentiment** (Figura 2.3). Questa visualizzazione chiarisce come, ad esempio, il sentiment negativo sia quasi interamente guidato da emozioni di *anger* e *disgust*, mentre quello positivo sia dominato dalla *joy*.

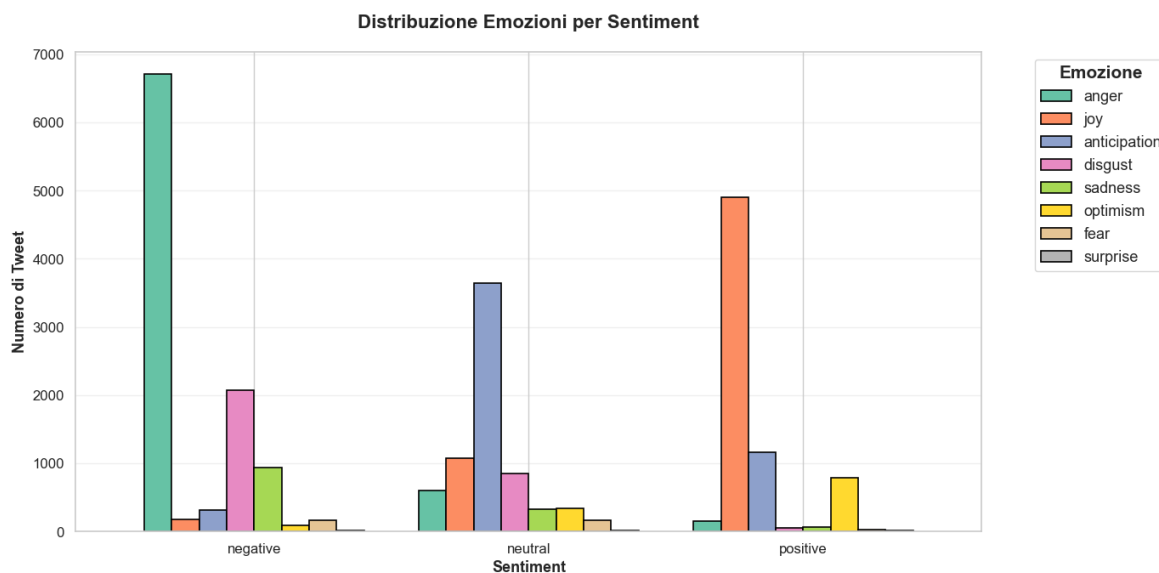


Figura 2.3: *Distribuzione delle emozioni segmentata per sentiment*

Dall'analisi della Figura 2.2 si evince un marcato sbilanciamento delle classi emozionali, che rende la loro classificazione un **task estremamente complesso** con i dati disponibili. Pertanto, l'analisi si è focalizzata esclusivamente sulla **classificazione ternaria** del sentiment (positivo, negativo, neutro).

2.3 Analisi dei termini con WordCloud

La **WordCloud** rappresenta uno strumento di visualizzazione estremamente efficace per l'analisi qualitativa dei contenuti testuali, consentendo di identificare immediatamente i termini più frequenti e rappresentativi di un corpus.

Sono state generate quattro WordCloud distinte: una **WordCloud generale** che aggrega tutti i tweet dell'intero dataset, offrendo una panoramica complessiva dei termini più ricorrenti, e tre WordCloud specifiche per sentiment, costruite esclusivamente sui tweet appartenenti a ciascuna categoria. Questa suddivisione consente di evidenziare le differenze semantiche e tematiche tra i diversi orientamenti di sentimento, facilitando l'interpretazione delle principali fonti di soddisfazione e insoddisfazione dei clienti Dell.

2.3.1 Metodologia di implementazione

Le WordCloud sono state generate attraverso un processo di preprocessing specifico implementato in Python. Il testo è stato inizialmente pulito rimuovendo **emoji** tramite la libreria `emoji`, seguito da **tokenizzazione avanzata** con `TweetTokenizer` di NLTK che preserva la struttura specifica dei tweet. È stata applicata un'estesa lista di **stopwords**, integrando quelle standard inglesi NLTK con un set custom di 45 parole non informative (retweet, ausiliari verbali, pronomi, ecc.) e termini come "dell" o "laptop" per la loro elevata frequenza ma bassa rilevanza in termini discriminativi. Il filtraggio ha eliminato sistematicamente **mentions** (@), **URL**, **hashtag** (convertiti senza #), **sequenze di punteggiatura**, **numeri puri** e parole inferiori a tre caratteri.

2.3.2 Visualizzazione dei risultati

La Figura 2.4 presenta la WordCloud generale costruita sull'intero corpus di tweet relativi a Dell. Questa evidenzia "service" come termine dominante (2175 occorrenze), seguito da "new", "support" e "customer", rivelando un focus predominante su assistenza clienti e prodotti innovativi. Termini come "time", "buy", "issue" e "warranty" sottolineano criticità post-acquisto. I temi principali, dunque, risultano essere: **supporto e servizio, problemi e tempi, prodotti/acquisti**.

Nel complesso, il sentiment appare misto ma inclinato al negativo, come dimostrato dalla distribuzione, con enfasi su customer service più che su innovazioni.

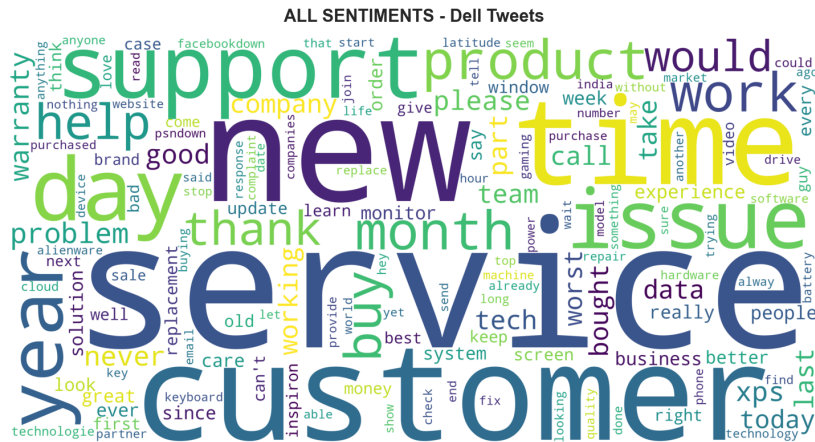
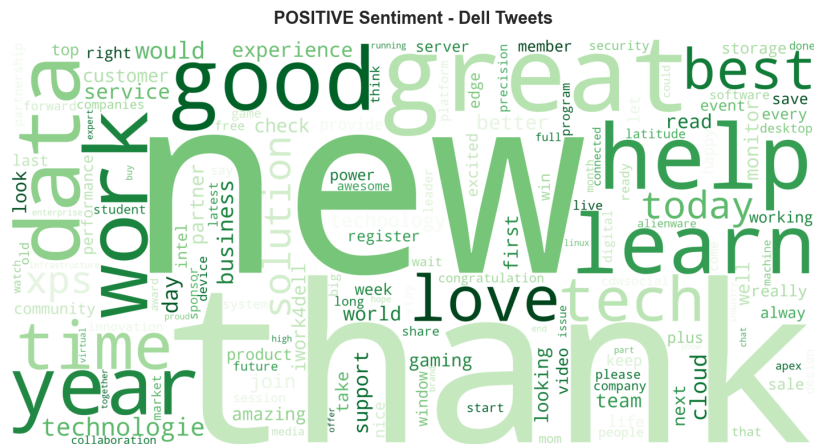


Figura 2.4: *WordCloud* generale dei termini

In Figura 2.5, sono mostrate le tre **WordCloud separate per sentiment** (positivo, negativo e neutrale). Offrono offrono un’analisi più raffinata e interessante rispetto all’aggregata, evidenziando come i temi Dell si distribuiscono emotivamente nei tweet.

Le tre WordCloud separate per sentiment sui tweet di Dell rivelano una chiara polarizzazione: dominano le critiche al servizio nei negativi, mentre i positivi lodano prodotti e apprendimento. In particolare l'analisi dettagliata è la seguente:

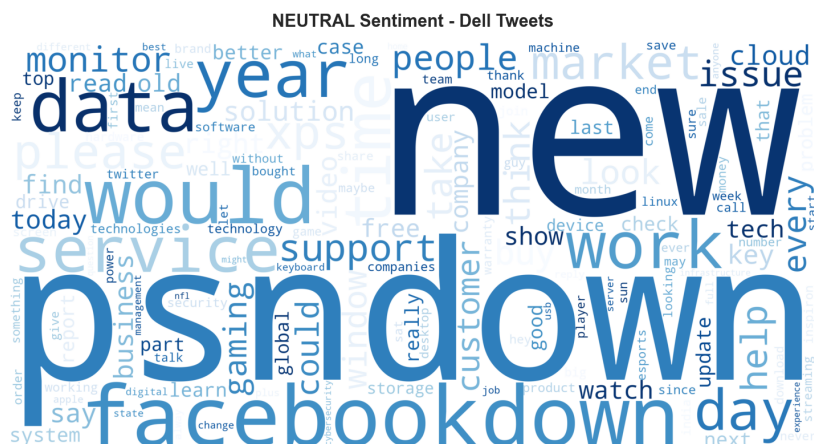
- **Sentiment Positivo:** Esprime elogi entusiasti per prodotti innovativi ("new", "XPS") e soddisfazione generale ("great", "love"), con gratitudine ("thank") e apprezzamento per il mondo tech.
- **Sentiment Negativo:** Dominato da critiche al servizio clienti, riflette frustrazioni per assistenza lenta ("days", "time") e problemi persistenti ("issue", "warranty"), con toni anche di delusione estrema ("never", "worst").
- **Sentiment Neutrale:** Focalizzato su eventi tecnici come outage ("psndown", "facebookdown") e discussioni oggettive su supporto, con richieste educate ("please") senza carica emotiva.



(a) WordCloud per il positive sentiment



(b) WordCloud per il negative sentiment



(c) WordCloud per il neutral sentiment

Figura 2.5: WordCloud separate per sentiment

3 Preprocessing ed embedding del testo

Questo capitolo descrive il processo di trasformazione del dataset preparato specificamente per il task di **classificazione del sentiment**. Il flusso è strutturato in fasi sequenziali per garantire dati puliti e rappresentazioni vettoriali significative, massimizzando l'accuratezza del modello. Tra le fasi principali ci sono: **split del dataset**, **tokenizzazione** e **pulizia** e **generazione di embedding con Word2Vec**.

3.1 Strategia di suddivisione del dataset

Prima di applicare le trasformazioni sul testo, è stato necessario definire una strategia di suddivisione del dataset rigorosa. Partendo da un totale di **24.970 tweet**, si è optato per uno split stratificato per garantire la rappresentatività statistica.

L'uso del parametro **stratify** garantisce che la distribuzione delle classi (Negative $\approx 42\%$, Positive $\approx 29\%$, Neutral $\approx 28\%$) rimanga identica in tutti i sottoinsiemi, evitando che il modello si alleni su una distribuzione diversa da quella di test.

La suddivisione è stata eseguita in due passaggi:

1. Separazione del **Test Set** (15%) dal totale.
2. Separazione del restante 85% in **Train Set** e **Validation Set**.

I numeri definitivi dei campioni sono:

- **Train Set:** 17.257 tweet (70%) → Per l'addestramento dei modelli e di Word2Vec.
- **Validation Set:** 3.686 tweet (15%) → Per il tuning degli iperparametri.
- **Test Set:** 3.696 tweet (15%) → Per la valutazione finale delle performance.

3.2 Pipeline di preprocessing

La funzione di pulizia è stata progettata per normalizzare il testo mantenendo le informazioni semantiche vitali. Le operazioni principali incluse nella pipeline sono:

- **Demojize:** Le emoji vengono convertite in stringhe di testo (es. il simbolo del sorriso diventa `:smile:`). Questo preserva l'informazione emotiva spesso cruciale nei tweet.
- **Tokenizzazione:** Utilizzo del `TweetTokenizer` (con `reduce_len=True` per accorciare parole come "cool" in "cool").

- **Filtraggio:**

- Rimozione delle menzioni (@user) per anonimizzare e ridurre il rumore.
- Rimozione dei link (http...).
- Gestione degli Hashtag: il simbolo # viene rimosso, ma la parola chiave viene mantenuta.
- Rimozione di punteggiatura e numeri isolati.

Di seguito il codice Python utilizzato per la trasformazione:

```
import emoji
from nltk.tokenize import TweetTokenizer
import string

def preprocess_for_word2vec(text):
    # Conversione emoji in testo
    text = emoji.demojize(text, delimiters=" ", " ")

    # Tokenizzazione (minuscolo + riduzione caratteri ripetuti)
    tknizr = TweetTokenizer(preserve_case=False, reduce_len=True)
    tokens = tknizr.tokenize(text)

    # Filtraggio
    cleaned_tokens = []
    for token in tokens:
        if token.startswith('@'): continue
        if token.startswith('http'): continue
        if token.startswith('#'):
            token = token[1:]
        if token in string.punctuation: continue
        if token.isdigit(): continue

        if token.strip():
            cleaned_tokens.append(token)

    return cleaned_tokens
```

3.3 Analisi Post-Processing

Le statistiche sul numero di token per tweet nei tre dataset confermano l'efficacia della pulizia:

- **Train:** Media Token = 23.17, Max = 70.

- **Validation:** Media Token = 23.21, Max = 60.
- **Test:** Media Token = 23.20, Max = 63.

Non sono stati rilevati tweet vuoti dopo il preprocessing. In Figura 3.1 si ha in alto parte di un tweet originale e in base l'effetto della pulizia e tokenizzazione. Si noti che tutto il testo è stato reso in minuscolo, i simboli # sono stati rimossi e le emoji sono state trasformate in stringhe di testo.

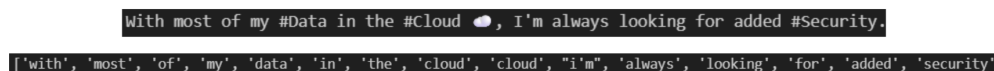


Figura 3.1: *Esempio di tokenizzazione di un tweet*

3.4 Rappresentazione vettoriale: Word2Vec

Una volta pulito il testo, il passaggio fondamentale per qualsiasi algoritmo di Machine Learning o Deep Learning è la **conversione** delle parole in **vettori numerici** (Embedding). Per questo progetto, è stato scelto di utilizzare **Word2Vec**, un modello che permette di mappare le parole in uno spazio vettoriale in modo tale che parole semanticamente simili si trovino vicine tra loro.

3.4.1 Addestramento del modello

Invece di utilizzare vettori pre-addestrati (come GloVe o GoogleNews), si è scelto di addestrare un modello Word2Vec **specifico sul Training Set**. Questa scelta è motivata dalla natura tecnica del dataset Dell: termini come "bios", "motherboard" o "driver" assumono significati e correlazioni specifiche in questo contesto che un modello generalista potrebbe non catturare.

La configurazione utilizzata tramite la libreria **gensim** è la seguente:

- **Vector Size (200):** Ogni parola viene rappresentata da un vettore di 200 numeri.
- **Window (3):** Il contesto considerato è di 3 parole prima e dopo la parola target.
- **Min Count (3):** Le parole che appaiono meno di 3 volte vengono ignorate per ridurre il rumore.

Successivamente sono stati fatti test per valutare la qualità degli embedding Word2Vec, analizzando la **similarità semantica** su alcune parole ed emoji. Ad esempio, per la

parola "good", il modello identifica "glorious" (0.473) e "surprising" (0.443) come semanticamente più correlate, mentre rileva anche "bad" (0.442) come termine vicino, suggerendo una buona cattura delle opposizioni semantiche.

3.4.2 Utilizzo nei modelli

Questa rappresentazione vettoriale costituisce la base comune per tutti i modelli descritti nel capitolo successivo:

1. **Per Naive Bayes, Logistic Regression e SVM:** Poiché questi modelli richiedono un input di dimensione fissa, è stata calcolata la media dei vettori di tutte le parole presenti in un tweet (*Mean Embedding*).
2. **Per LSTM e Bi-LSTM:** Le reti neurali utilizzano la sequenza completa dei vettori parola per parola, mantenendo l'ordine temporale e costruendo una matrice di embedding dedicata.

Questo approccio garantisce che tutti i modelli confrontati beneficino della stessa comprensione semantica del vocabolario, sfruttando il medesimo embedding.

4 Modelli testati

In questo capitolo vengono presentati i cinque approcci di classificazione implementati per il task di Sentiment Analysis: **Naive Bayes**, **Logistic Regression**, **Support Vector Machine (SVM)**, **Long Short-Term Memory (LSTM)** e **Bidirectional Long Short-Term Memory (Bi-LSTM)**. Tutti i modelli utilizzano come input le rappresentazioni vettoriali generate da Word2Vec (descritte nel Capitolo 3), garantendo così un confronto equo basato sulla stessa comprensione semantica del testo.

4.1 Modello 1: Naive Bayes

È stato utilizzato un **Gaussian Naive Bayes** come primo classificatore per l'analisi del sentiment sui tweet relativi a Dell. Questo modello probabilistico, basato sull'assunzione di indipendenza condizionale tra le feature, si distingue per la sua semplicità computazionale, rapidità di addestramento e robustezza su dataset di dimensioni moderate, rendendolo ideale come baseline per confrontare approcci più complessi.

4.1.1 Configurazione

Per l'implementazione del Gaussian Naive Bayes è stata utilizzata la libreria **scikit-learn**, lasciando tutti i parametri al loro valore di default per mantenere una configurazione standard e riproducibile come baseline.

4.1.2 Risultati

I risultati del Gaussian Naive Bayes mostrano un'alta velocità di training (solo 0.08 secondi), ma una moderata performance con un'**accuracy** del **60.58%** sul test set, confermando la sua efficacia come baseline rapida.

La matrice di confusione del Gaussian Naive Bayes, mostrata in Figura 4.1, rivela risultati moderati, mostrando un'eccellente capacità di identificare i tweet negativi, mentre i positivi raggiungono un discreto 46.4% con buona precision. Tuttavia emergono **problemi significativi sui neutrali**. Questo pattern, tipico di Naive Bayes su dati sbilanciati con feature indipendenti, evidenzia la difficoltà nel cogliere sfumature semantiche, suggerendo la necessità di algoritmi più sofisticati per migliorare le performance complessive.

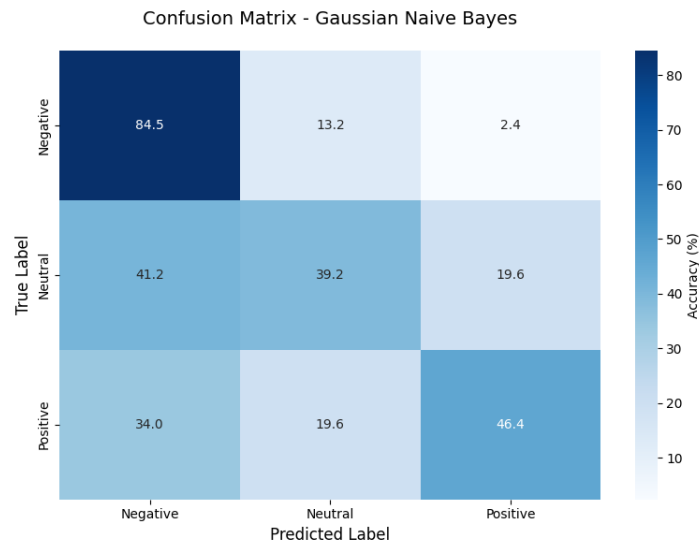


Figura 4.1: *Matrice di confusione del Gaussian Naive Bayes*

4.2 Modello 2: Logistic Regression

Come secondo classificatore è stata adottata la **Logistic Regression**, un modello lineare probabilistico che ottimizza i pesi delle feature tramite regressione logistica multinomiale, offrendo maggiore capacità espressiva rispetto a Naive Bayes pur mantenendo efficienza computazionale e interpretabilità dei coefficienti.

4.2.1 Configurazione

Il modello è stato implementato utilizzando la libreria `scikit-learn` con i seguenti parametri:

- **Solver:** `lbfgs` (ottimizzato per problemi multiclasse).
- **Max Iterations:** 1000 (per garantire la convergenza).
- **Random State:** 42 (per la riproducibilità).

Poiché la Logistic Regression richiede un input a dimensione fissa, i vettori delle singole parole di un tweet sono stati aggregati calcolandone la media (*Mean Embedding*).

4.2.2 Risultati

Il training è stato rapido, completandosi in soli **0.60 secondi**. Le performance in termini di **accuracy** sono aumentate, raggiungendo un valore di **71.81%** con il test set.

La matrice di confusione sul test set, mostrata in Figura 4.2, evidenzia migliori

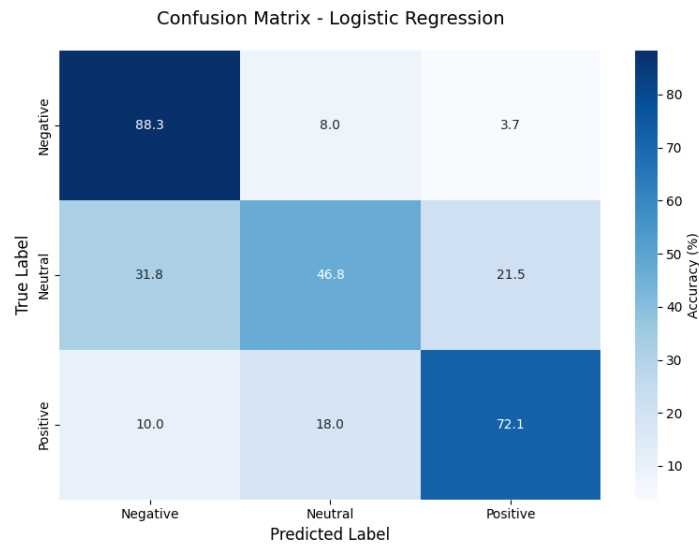


Figura 4.2: *Matrice di confusione della Logistic Regression*

risultati rispetto al Naive Bayes, riuscendo maggiormente a bilanciare le classi. Sono rimasti, però, dei problemi con la classe Neutral che colleziona solamente il 46.8% di predizioni corrette.

4.3 Modello 3: Support Vector Machine (SVM)

Per catturare relazioni non lineari tra le feature degli embedding Word2Vec, è stato implementato un classificatore **SVM con kernel RBF (Radial Basis Function)**. Questo kernel gaussiano mappa i dati in uno spazio ad alta dimensionalità, consentendo separazioni complesse e migliorando la generalizzazione rispetto a Logistic Regression.

4.3.1 Configurazione

Il modello è stato implementato utilizzando la libreria `scikit-learn` con i seguenti parametri:

- **Kernel:** `rbf` (per gestire la non linearità).
- **Class Weight:** `balanced` (per penalizzare maggiormente gli errori sulle classi meno rappresentate).
- **Gamma:** `scale`.

Da notare che per questo modello si è deciso di provare a bilanciare le classi grazie al parametro `class weight`, poichè i precedenti risultati avevano fatto osservare un problema proprio in questo senso.

4.3.2 Risultati

Il tempo di addestramento è aumentato drasticamente a **23.39 secondi**, riflettendo la maggiore complessità computazionale dell'algoritmo. Tuttavia, questo costo è giustificato da un netto miglioramento delle performance, totalizzando un'accuracy con il test set di **76.00%**.

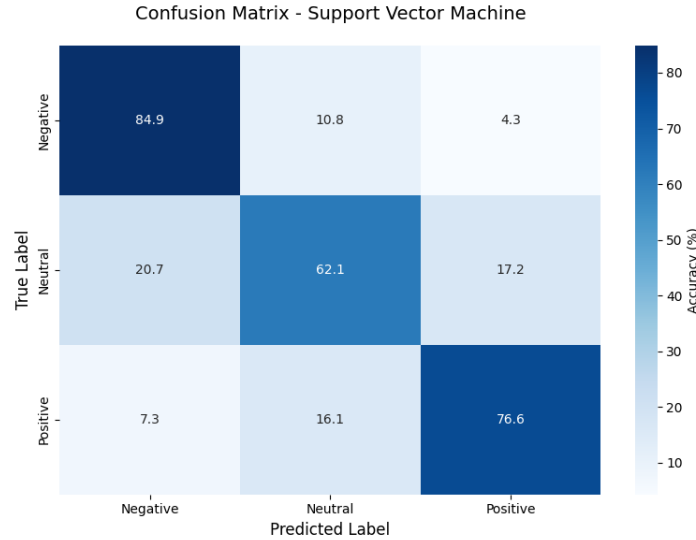


Figura 4.3: *Matrice di confusione del Support Vector Machine*

Anche la matrice di confusione rappresentata nella Figura 4.3 conferma un ulteriore affinamento. Tra i miglioramenti chiave si nota che il 62.1% della classe Neutral viene predetta correttamente, totalizzando circa un **+15%** rispetto alla Logistic Regression, confermando che il bilanciamento delle classi ha portato grandi benefici. L'SVM eccelle nel separare confini non lineari degli embedding, rendendolo il baseline classico dei modelli Machine Learning più efficace per questo task.

4.4 Modello 4: Long Short-Term Memory (LSTM)

Approfondendo l'analisi, per superare i limiti dei classificatori classici, è stato implementato un modello **LSTM (Long Short-Term Memory)**, una rete neurale ricorrente progettata per catturare dipendenze sequenziali e contestuali nel testo. L'architettura LSTM utilizza porte di memoria per regolare selettivamente il flusso di informazioni attraverso la sequenza di embedding Word2Vec, preservando contesto a lungo termine che Naive Bayes/SVM ignorano. Questo la rende ideale per il sentiment analysis su tweet brevi ma espressivi.

4.4.1 Configurazione

Il modello è stato implementato utilizzando la libreria **keras**. Il modello richiede che le label siano convertite in sequenze numeriche, ciò è stato fatto utilizzando **LabelEncoder**. Successivamente, è stata creata una matrice di embedding utilizzando i vettori **Word2Vec** già calcolati e zeri per parole "Out of Vocabulary". Infine, sono stati calcolati i **Class weights** per bilanciare le classi e impostato un **dropout** multiplo per prevenire l'overfitting. I principali iperparametri della rete sono:

- **Optimizer:** Adam.
- **Learning rate:** 0.0005.
- **Activation function:** softmax.
- **Numero di epoche:** 20 con **patience** di 3 (se per 3 epoche non migliora si stoppa).

4.4.2 Risultati

Il training ha avuto un "early stopping" all'epoca 14, registrando dunque come **epoca migliore la 11**. Nella Figura 4.4 si possono osservare le statistiche di loss e accuracy per il training e validation set, per visualizzare l'andamento dell'addestramento.

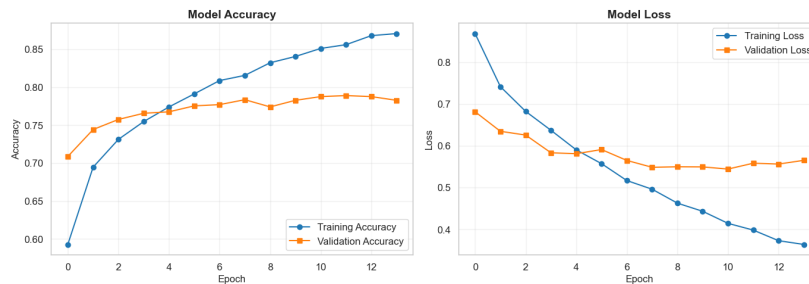


Figura 4.4: Andamento delle curve di loss e accuracy per il modello LSTM

La curva di training mostra **convergenza stabile** con una loss che decade senza overfitting grazie al **dropout** e **early stopping**.

Le performance, in termini di accuracy, sono molto positive, totalizzando un valore di **79.00%** sul test set. Questo è un indice che l'utilizzo di reti neurali è indicato per il task di sentiment analysis e la scelta degli iperparametri ha aiutato il modello a generalizzare bene sul dataset.

La matrice di confusione, riportata in Figura 4.5, conferma le migliori performance complessive, consolidando i guadagni dati dalla cattura del contesto sequenziale sui tweet Dell. Aumenta anche il **recall** sulla classe **Neutral**, facendo un +6% rispetto al modello SVM; ciò conferma che il modello a rete neurale ha performance più solide anche con la classe più complicata a livello semantico e meno frequente nel dataset.

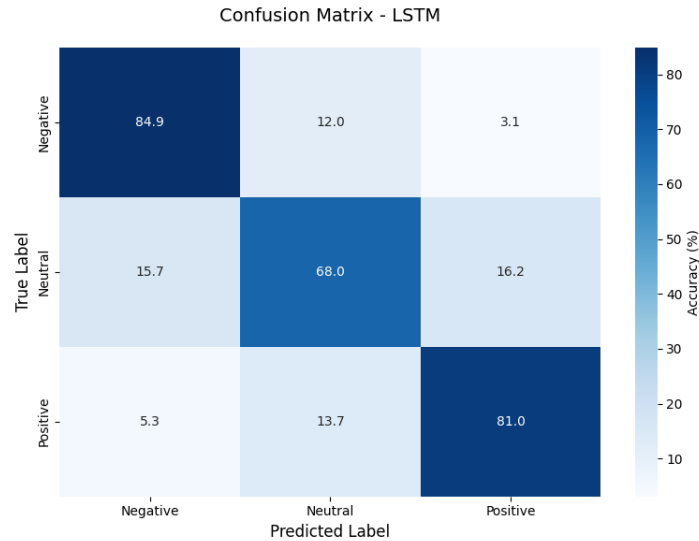


Figura 4.5: Matrice di confusione del modello LSTM

4.5 Modello 5: Bidirectional Long Short-Term Memory (Bi-LSTM)

Per spingere ulteriormente le performance oltre l'LSTM unidirezionale, è stato implementato un **Bi-LSTM** (Bidirectional Long Short-Term Memory), evoluzione architetturale delle RNN che processa le sequenze di embedding Word2Vec contemporaneamente in entrambe le direzioni: **forward** e **backward**.

Questa bidirezionalità genera rappresentazioni contestuali più ricche concatenando le hidden state delle due LSTM indipendenti, catturando dipendenze bilaterali che l'unidirezionale trascura.

4.5.1 Configurazione

Il modello è stato implementato utilizzando la libreria **keras**. Il processo di trasformazione delle label e degli embedding è stato il medesimo di quello descritto per la rete LSTM. Anche in questo caso, sono stati utilizzati i **Class weights** per bilanciare le classi e impostato un **dropout** multiplo per prevenire l'overfitting. I principali iperparametri della rete sono:

- **Optimizer:** Adam.
- **Learning rate:** 0.0005¹.
- **Activation function:** softmax.

¹In più è stata aggiunta la callback **ReduceLROnPlateau** che dimezza il learning rate se per due epoche non c'è miglioramento nella loss di validazione. Questo porta a un maggiore fine-tuning.

- **Numero di epoche:** 20 con **patience** di 5 (se per 5 epoche non migliora si stoppa).

4.5.2 Risultati

Il training, anche in questo caso, ha subito un "early stopping" all'epoca 12, registrando dunque come **epoca migliore la 7**. Nella Figura 4.6 si possono osservare le statistiche di loss e accuracy per il training e validation set, per visualizzare l'andamento dell'addestramento.

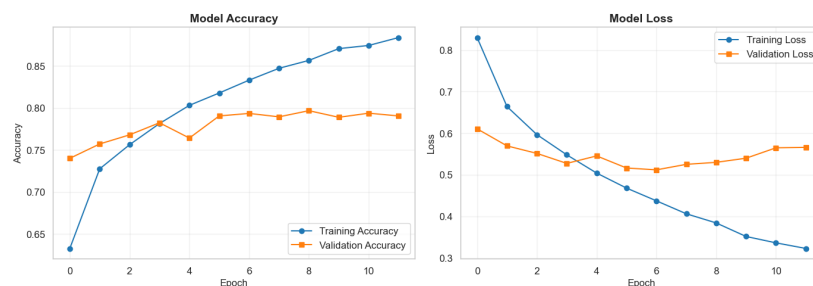


Figura 4.6: Andamento delle curve di loss e accuracy per il modello Bi-LSTM

Le **curve di training** del Bi-LSTM mostrano una convergenza solida. Si nota un **lieve overfitting**, ma efficacemente mitigato da "early stopping" che ha selezionato automaticamente l'epoca ottimale, evitando degradazione ulteriore e garantendo i migliori risultati possibili (80% accuracy in validation) come massimo realistico per questa architettura sui tweet Dell.

Le performance, in termini di accuracy, sono le migliori registrate per tutti i modelli, con un valore di **79.84%** sul test set. Questo indica che il modello bidirezionale coglie meglio il contesto semantico del tweet, raggiungendo risultati ottimali per un task complicato di sentiment analysis.

La matrice di confusione del Bi-LSTM, mostrata nella Figura 4.7, conferma **performance simili all'LSTM unidirezionale**, con un plateau raggiunto ma piccoli miglioramenti tangibili sui negativi grazie al contesto bidirezionale. Gli errori sui neutrali rimangono stabili e i positivi consistenti, evidenziando come il guadagno marginale rifletta i **limiti intrinseci** del dataset più che architetturali, dove il Bi-LSTM raffina senza rivoluzionare.

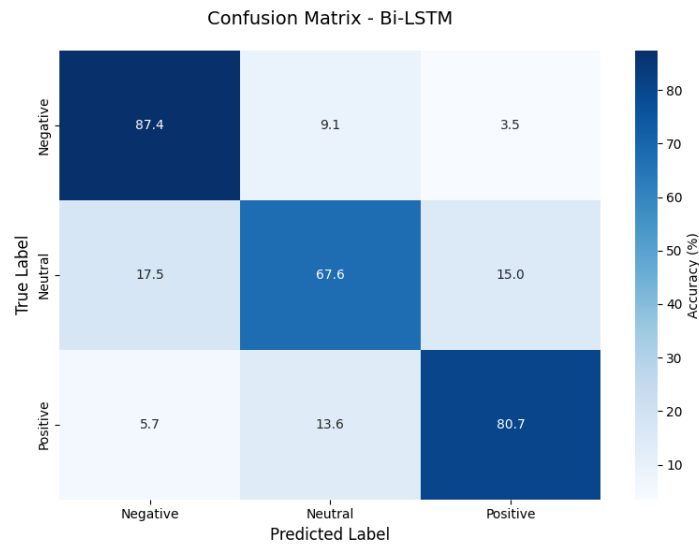


Figura 4.7: *Matrice di confusione del modello Bi-LSTM*

4.6 Analisi finale e conclusione

I cinque modelli testati per la classificazione del sentiment sui tweet Dell mostrano una progressione chiara dalle **baseline classiche** alle **reti neurali sequenziali**.

Naive Bayes stabilisce un baseline rapido ma limitato (circa 60% accuracy), superato nettamente da **Logistic Regression** e **SVM** che raggiungono più del 70% sfruttando meglio le feature Word2Vec lineari/non lineari. L'**LSTM** segna il breakthrough catturando contesto sequenziale, mentre il **Bi-LSTM** conferma il plateau con affinamenti marginali, principalmente sulle classi minoritarie.

Tutti i modelli eccellono sui negativi (84-87%, dominanti per "service/support"), migliorano gradualmente sui positivi, ma lottano con i neutrali che sono fattualmente ambigui e meno rappresentati nel dataset. Le confusion matrix rivelano una **sovra-classificazione persistente dei neutrali verso negativi**, tipica di dataset sbilanciati.

Il Bi-LSTM emerge vincitore, massimizzando il potenziale degli embedding Word2Vec con architetture sequenziali. Il plateau raggiunto (circa 80% di accuracy) suggerisce **limiti dati-driven** più che modellistici: rumorosità dei tweet, ambiguità semantica e sbilanciamento.

5 Information Extraction

L'*Information Extraction (IE)* è una branca del *Natural Language Processing (NLP)* definita come l'insieme di metodologie e tecniche volte all'identificazione e all'estrazione automatica di informazioni rilevanti da testi non strutturati.

In particolare, gli obiettivi principali dell'*IE* sono:

- la conversione dei contenuti testuali in dati strutturati, rendendoli analizzabili e utilizzabili per successive elaborazioni computazionali;
- l'individuazione selettiva di elementi informativi specifici, quali entità, relazioni o concetti chiave, piuttosto che la comprensione globale del testo.

In questo contesto, uno dei principali task dell'*Information Extraction* è la *Key-Phrase Extraction (KPE)*, una tecnica di analisi testuale volta all'estrazione automatica di parole o espressioni multi-token che rappresentano i concetti più significativi di un documento. In altre parole, le *keyphrase* sono una sintesi del testo contenente solo le espressioni maggiormente rappresentative del contenuto.

Un ulteriore task fondamentale dell'*IE* è la *Named Entity Recognition (NER)*, una procedura di identificazione e classificazione automatica delle entità testuali all'interno di categorie predefinite, quali persone, organizzazioni, luoghi, date, quantità o altre tipologie semantiche.

In conclusione, nei paragrafi seguenti si svolgono i task di *Key-Phrase Extraction (KPE)* e *Named Entity Recognition (NER)*, al fine di estrarre e strutturare le informazioni rilevanti contenute nel corpus oggetto di studio.

5.1 *Key-Phrase Extraction (KPE)*

L'attività di *Key-Phrase Extraction (KPE)* sul dataset *Sentiment & Emotions Labelled Tweets* si articola in tre fasi principali. In primo luogo, si applica un iter di pre-processing dei tweet, volto a normalizzare il testo tipico dei contenuti social. Successivamente, si esegue l'estrazione delle keyphrase mediante le librerie *spaCy* e *textacy*, utilizzate come strumenti per l'individuazione automatica delle espressioni principali. Infine, si analizzano e discutono i risultati sperimentali ottenuti.

5.1.1 Pre-processing

Nell'ambito della *Key-Phrase Extraction (KPE)*, la fase di pre-processing del testo incide direttamente sulla qualità delle rappresentazioni testuali e, di conseguenza, sull'efficacia dei metodi di estrazione.

In particolare, il pre-processing persegue tre obiettivi principali:

- la riduzione del rumore testuale, particolarmente rilevante nei contenuti generati sui social media;
- la normalizzazione delle varianti linguistiche, al fine di limitare la frammentazione lessicale;
- la preservazione dell'informazione semanticamente rilevante ai fini dell'identificazione delle espressioni chiave.

Tali operazioni assumono particolare importanza nel caso dei tweet, caratterizzati da brevità, informalità, elevata densità di simboli non linguistici (ad esempio hashtag, menzioni, emoji) e frequente utilizzo di abbreviazioni o forme contratte. Dunque, senza un'adeguata fase di normalizzazione, questi elementi possono introdurre rumore e sparsità lessicale, compromettendo i meccanismi di ranking e selezione delle keyphrase.

Alla luce di tali considerazioni, sul dataset *Sentiment & Emotions Labelled Tweets* è stata implementata la seguente pipeline di pre-processing dei tweet:

- **Lowercasing:** Tutti i tweet vengono convertiti in formato lower-case al fine di uniformare le varianti ortografiche della medesima unità lessicale (ad esempio, *Dell*, *DELL*, *dell*). Tale operazione riduce la frammentazione del vocabolario e la sparsità dei dati, senza comportare una perdita di informazione semantica.
- **Rimozione degli URL:** I collegamenti ipertestuali (<http>, <https>, www) non apportano contenuto semantico rilevante ai fini dell'identificazione delle keyphrase e introducono token artificiali indesiderati. Per tale motivo, vengono rimossi dal testo.
- **Espansione delle contrazioni e abbreviazioni:** I tweet presentano frequentemente contrazioni standard (ad es. *can't*, *I'll*) e abbreviazioni informali tipiche del linguaggio dei social media (ad es. *ty*, *pls*, *idk*). In assenza di espansione, espressioni semanticamente equivalenti (ad esempio, *can't login* e *cannot login*) verrebbero trattate come pattern distinti. Poiché la *KPE* si basa su frequenze, co-occorrenze o strutture fondate su grafi, l'espansione contribuisce a ridurre la sparsità lessicale e a migliorare la coerenza semantica.
- **Conversione delle emoji in testo descrittivo:** Le emoji costituiscono segnali semantici non trascurabili nei contenuti social, quindi la loro rimozione comporta una perdita informativa; pertanto, esse vengono convertite in descrizioni testuali mediante la funzione `emoji.demojize`. In tal modo, le emoji diventano token interpretabili dal modello e possono contribuire ai meccanismi di estrazione delle keyphrase.
- **Rimozione delle menzioni (@):** Le menzioni di utenti raramente rappresentano il contenuto tematico (ovvero il topic) del tweet. La loro rimozione consente quindi di ridurre il rumore.

- **Gestione degli hashtag (#):** Il simbolo # viene rimosso mantenendo tuttavia il testo dell'hashtag, poiché quest'ultimo spesso contiene informazioni tematiche rilevanti. Tale scelta consente di preservare il contenuto semantico evitando al contempo la creazione di token non standard.
- **Rimozione di bullet point e simboli grafici speciali:** Elementi grafici quali bullet point o simboli decorativi non apportano informazione linguistica e vengono pertanto eliminati per ridurre il rumore testuale.
- **Rimozione di caratteri non testuali:** Vengono mantenuti caratteri alfanumerici, spazi e alcuni simboli funzionali, mentre i caratteri non linguistici o rumorosi vengono rimossi. Tale operazione previene la generazione di token anomali che potrebbero interferire con le fasi successive di tokenizzazione e analisi.
- **Normalizzazione degli spazi:** Gli spazi multipli vengono compressi in un singolo spazio e gli spazi iniziali o finali eliminati, al fine di garantire una rappresentazione testuale coerente ed evitare la creazione di token vuoti.

5.1.2 *KPE* mediante *spaCy* e *textacy*

Uno degli strumenti open-source più diffusi per l'implementazione di task *NLP* è *spaCy*, una libreria progettata per l'elaborazione testuale sistematica.

In particolare, in *spaCy*, il processo di analisi automatica del testo avviene mediante l'impiego di modelli linguistici pre-addestrati che operano all'interno della seguente pipeline modulare.

- **Tokenizer:** Componente che segmenta il testo in unità elementari (*token*), quali parole, segni di punteggiatura e simboli. La tokenizzazione costituisce il primo passo dell'analisi poiché tutte le successive annotazioni linguistiche vengono effettuate a livello di *token*.
- **POS Tagger:** Componente che assegna a ciascun *token* una categoria grammaticale (Part-of-Speech), come sostantivo, verbo, aggettivo, ecc. L'informazione di POS è fondamentale per numerosi task di estrazione, in quanto consente di filtrare o selezionare specifiche classi lessicali.
- **Lemmatizer:** Componente che associa a ciascun *token* la sua forma base (lemma), permettendo di ricondurre varianti flessionali alla stessa rappresentazione canonica (ad esempio, *running* → *run*). Ciò contribuisce a ridurre la frammentazione del vocabolario.
- **Dependency Parser:** Componente che analizza le relazioni grammaticali tra i *token*, producendo una struttura ad albero che rappresenta le dipendenze sintattiche. Tale struttura consente, ad esempio, l'identificazione automatica di *noun chunks*, utilizzati successivamente come sottoinsieme iniziale per la scelta delle keyword più rappresentative.

- **NER (Named Entity Recognizer):** Componente che identifica e classifica entità nominate presenti nel testo (ad esempio persone, organizzazioni, luoghi, date), assegnando loro etichette semantiche predefinite.

In sintesi, nell'ambito della *Key-Phrase Extraction*, *spaCy* svolge un ruolo centrale nella fase di analisi linguistica preliminare, infatti:

- la tokenizzazione e l'annotazione forniscono la struttura di base del testo;
- la lemmatizzazione permette di normalizzare le varianti morfologiche, riducendo la sparsità lessicale.
- il dependency parsing consente di individuare unità sintattiche che possono essere utilizzate come candidati per l'estrazione delle espressioni chiave;

In questo contesto, le funzionalità di *spaCy* vengono ulteriormente estese mediante l'impiego di *textacy*, una libreria che opera direttamente sugli oggetti *Doc* generati dalla pipeline di *spaCy*, sfruttandone le annotazioni per implementare diversi algoritmi di *Key-Phrase Extraction*, tra cui metodi di natura statistica e *graph-based*, come *TextRank*.

In particolare, *TextRank* è un algoritmo non supervisionato che, applicato al testo, costruisce un grafo in cui i nodi rappresentano unità lessicali (tipicamente *token* filtrati in base alla categoria grammaticale), mentre gli archi rappresentano relazioni di co-occorrenza tra *token* all'interno di una finestra scorrevole nel testo.

Una volta costruito il grafo, viene applicato un algoritmo iterativo di tipo *PageRank* che assegna a ciascun nodo un punteggio di importanza basato sulla struttura delle connessioni: un nodo è considerato rilevante se è connesso ad altri nodi a loro volta rilevanti. I punteggi convergono attraverso iterazioni successive fino alla stabilizzazione. In sintesi, nel contesto della *KPE*, i *token* con punteggio più elevato vengono selezionati e, ove opportuno, combinati in espressioni *multi-token* per costituire le keyphrase finali. L'intero processo è non supervisionato e non richiede dati annotati manualmente.

In conclusione, in tale configurazione, *spaCy* fornisce l'infrastruttura di annotazione, mentre *textacy* implementa i meccanismi di costruzione del grafo e di ranking delle unità lessicali, rendendo il processo di *Key-Phrase Extraction* riproducibile e metodologicamente fondato.

5.1.3 Pipeline di *KPE*: confronto tra modelli *spaCy*

In questa sezione vengono descritte due configurazioni sperimentali per la *Key-Phrase Extraction* basate sull'utilizzo congiunto delle librerie *spaCy* e *textacy*. In particolare, si confrontano un modello linguistico leggero (`en_core_web_sm`) e un modello basato su architettura Transformer (`en_core_web_trf`), evidenziandone le differenze operative.

KPE con spaCy en_core_web_sm e textacy

La seguente pipeline implementativa descrive il metodo di estrazione delle keyphrase mediante il modello `en_core_web_sm` e l'algoritmo *TextRank*.

- **Caricamento del modello linguistico `en_core_web_sm`:** si carica il modello `en_core_web_sm`, il più piccolo e veloce modello pre-addestrato per la lingua inglese della libreria *spaCy*, che fornisce le principali annotazioni linguistiche necessarie all'analisi automatica del testo, tra cui: tokenizzazione, lemmatizzazione, POS tagging, dependency parsing e Named Entity Recognition.

In particolare, tale modello è necessario poiché *textacy* opera sugli oggetti `Doc` generati da *spaCy*, sfruttandone le annotazioni linguistiche per eseguire analisi testuale.

- **Definizione della funzione di estrazione delle keyphrase:** la funzione `extract_keyphrases` opera su tre parametri di input:
 - `text`: il testo in input da sottoporre ad analisi;
 - `n`: il numero massimo di keyphrase da estrarre (*spaCy* propone un valore predefinito pari a `n=5`);
 - `aggregate`: parametro booleano che consente l'aggregazione di varianti simili delle keyphrase.

In particolare, tali parametri vengono inclusi nella seguente pipeline:

- **Creazione dell'oggetto `Doc`:** il testo in input viene convertito in un oggetto `Doc` mediante l'istruzione `textacy.make_spacy_doc`, dunque contiene tutte le annotazioni linguistiche prodotte dal modello `en_core_web_sm`.
- **Estrazione delle keyphrase tramite *TextRank*:** l'estrazione delle keyphrase avviene tramite il comando `textacy.extract.keyterms.textrank(doc, normalize="lemma", t=n)` che svolge le seguenti funzioni:
 - * Il parametro `normalize = "lemma"` indica che i termini candidati vengono normalizzati alla loro forma lemmatizzata, riducendo la variabilità morfologica.
 - * Il parametro `t = n` limita l'output alle prime N keyphrase più rilevanti.
 - * l'algoritmo *TextRank* segue il paradigma *graph-based* per l'estrazione di termini chiave, cioè: costruisce un grafo dei termini candidati, poi assegna a ciascun nodo un punteggio di importanza (weight) mediante *PageRank*, infine i termini vengono ordinati in base al punteggio in ordine decrescente e si restituiscono i primi N termini.

In questo modo, la funzione restituisce una lista di tuple nella forma $(term, weight)$, dove *term* rappresenta la keyphrase estratta, mentre *weight* ne indica il punteggio di importanza.

- **Aggregazione delle varianti:** si uniscono varianti morfologiche o forme simili di uno stesso termine (ad esempio singolare e plurale), riducendo la ridondanza nell'insieme delle keyphrase restituite.
- **Estrazione dei *noun chunks*:** oltre alle keyphrase, la funzione restituisce i *noun chunks*, ossia l'insieme dei token fondamentali individuati mediante il dependency parsing fornito da *spaCy*.

KPE con spaCy en_core_web_trf e textacy

La pipeline descritta in questa seconda configurazione utilizza il modello Transformer `en_core_web_trf`, introducendo un metodo di selezione controllata dei candidati e una fase di aggregazione delle keyphrase cross-tweet.

- **Caricamento del modello Transformer `en_core_web_trf`:** si carica `en_core_web_trf`, un modello *spaCy* per la lingua inglese basato su architettura Transformer che genera rappresentazioni contestuali profonde (ossia rappresentazioni che tengono conto del contesto in cui ciascun token appare nel testo). In particolare, *textacy* utilizza tale modello per generare oggetti `Doc`.
- **Estrazione delle keyphrase tramite *TextRank*:** si utilizza una funzione di estrazione analoga al caso descritto per il modello `en_core_web_sm`, ma con uno step aggiuntivo riguardante la definizione dell'insieme dei candidati.

Nello specifico, l'insieme dei candidati non coincide con tutti i termini del grafo di *TextRank*, ma è limitato ai soli sintagmi nominali composti da almeno due parole. Dal punto di vista operativo, questo consente di escludere espressioni nominali monolemmatiche, comportando i seguenti vantaggi:

- diminuzione della probabilità di inclusione di parole ad alta frequenza ma bassa specificità;
- concentrazione dell'analisi su unità lessicali composte al fine di aggiungere specificità semantica, ad esempio: `model`, `system`, `data` sono semanticamente più vaghi rispetto a `language model` o `data analysis`.

In sintesi, la scelta di selezionare candidati tra sintagmi nominali composti da almeno due *token* rappresenta un trade-off tra precisione e copertura, dal momento che escludere sintagmi di lunghezza unitaria significa potenzialmente perdere concetti rilevanti espressi con un solo termine a favore di una maggiore interpretabilità dei risultati.

- **Aggregazione cross-tweet:** in questa fase l'analisi cambia scala dal momento che non si considera più il singolo tweet, bensì l'intero corpus. In particolare, l'obiettivo è la generazione di una classifica delle keyphrase più ricorrenti, che rappresenta una sintesi quantitativa dei temi predominanti nel dataset analizzato. Nello specifico, gli step implementativi sono:
 - **Flatten delle keyphrase:** le keyphrase estratte sono inizialmente organizzate come liste annidate, cioè come una lista per ciascun tweet. Dunque, l'operazione di flattening consiste nel convertire tale struttura $[[kp_1, kp_2], kp_3, [kp_4, kp_5]]$ in una lista lineare $[kp_1, kp_2, kp_3, kp_4, kp_5]$. In questo modo si ottiene un unico insieme contenente tutte le keyphrase estratte dall'intero corpus.
 - **Aggregazione delle varianti a livello corpus:** successivamente, si applica una procedura di aggregazione delle varianti lessicali sull'insieme complessivo delle keyphrase al fine di ricondurre forme simili a una rappresentazione canonica unica.
 - **Conteggio delle frequenze globali:** infine, si conta il numero di occorrenze di ciascuna keyphrase nell'intero corpus al fine di quantificarne la rilevanza globale.

5.1.4 Risultati sperimentali *KPE*

La Tabella 5.1 mostra i tweet analizzati, mentre i risultati sperimentali derivanti dall'applicazione dei due modelli *spaCy* sul dataset sono descritti in Tabella 5.2.

Testi dei tweet di riferimento
whatif qwerty were modified for programmers so things like brackets parens quotes operators etc. moved to the middle near g-h splitting left right brackets to separate hands and relieving the less dextrous pinky finger
she is getting a new laptop when the one she has one only 2-3 years old. more than triple the price - though much higher utility.
as celebrates its 40th anniversary bob faletta and spoke with me about the milestones companies and personalities that helped build the channel. crn40 podcast internationalpodcastday
your customer service is horrible especially agent syedfaisal who has made this experience of purchasing a new computer downright awful and i will reconsider ever buying a dell in the future
give the man what he paid for

Tabella 5.1: *Tweet di riferimento per il confronto tra modelli spaCy.*

spaCy en_core_web_sm	
Noun Chunks	Keyphrases
whatif qwerty programmers things brackets operators middle g-h splitting right brackets hands less dextrous pinky finger	dextrous pinky finger right bracket whatif qwerty bracket paren h splitting
she new laptop more than triple the price - though much higher utility	high utility new laptop year old price
as celebrates its 40th anniversary bob faletra me milestones companies personalities that channel crn40 podcast	crn40 podcast internationalpodcastday 40th anniversary bob faletra milestone company personality channel
your customer service horrible especially agent syedfaisal who experience new computer i dell future	customer service agent syedfaisal new computer horrible awful
man what he	man

spaCy en_core_web_trf	
Noun Chunks	Keyphrases
brackets parens quotes operators left right brackets less dextrous pinky finger	less dextrous pinky finger left right brackets
new laptop more than triple the price much higher utility	more than triple the price new laptop
its 40th anniversary crn40 podcast	crn40 podcast
your customer service especially agent syedfaisal new computer	especially agent syedfaisal your customer service new computer
\	\

Tabella 5.2: Confronto tra noun chunks e keyphrases estratte con spaCy en_core_web_sm e spaCy en_core_web_trf.

In particolare, dall'analisi dei risultati riportati in Tabella 5.2 emergono le seguenti differenze tra i modelli *spaCy en_core_web_sm* e *en_core_web_trf*.

- **Ampiezza e granularità dei *noun chunks*:**

- **en_core_web_sm:** il modello produce chunk sintatticamente rumorosi caratterizzati da pronomi (**he**, **she**, **me**, **who**, **i**), elementi poco informativi (**things**, **that**, **what**) e sintagmi nominali di lunghezza unitaria (**man**).

- **en_core_web_trf**: il modello genera un numero inferiore di chunk caratterizzati da strutture nominali più compatte e maggiore coerenza semantica.
- **Qualità delle keyphrases:**
 - **en_core_web_sm**: il modello produce molteplici keyphrase generiche (**price**, **personality**, **channel**), elabora diversi aggettivi valutativi che non sono concetti nominali strutturati (**horrible**, **awful**), e tende a frammentare concetti composti (**price** piuttosto che **more than triple the price**).
 - **en_core_web_trf**: il modello riduce significativamente il numero di keyphrase selezionando solo unità lessicali *multi-token* rappresentanti concetti semanticamente densi rispetto ad un nucleo nominale (**more than triple the price**, **especially agent syedfaisal**).
- **Impatto del vincolo ≥ 2 *token*:** il vincolo introdotto nel modello **en_core_web_trf** produce tre effetti principali:
 - **Riduzione del rumore lessicale:** vengono esclusi termini generici o poco informativi;
 - **Maggiore specificità semantica:** le espressioni *multi-token* catturano meglio concetti completi;
 - **Perdita di copertura in casi brevi:** nell'ultimo tweet (**give the man what he paid for**), il modello **en_core_web_trf** non produce output, mentre **en_core_web_sm** estrae **man**. In sintesi, **en_core_web_sm** privilegia la copertura e restituisce un insieme più ampio di candidati, ma con maggiore rumore e frammentazione concettuale.

Nel seguito, si presenta un'analisi quantitativa finalizzata alla valutazione delle keyphrase generate dai modelli *spaCy* adottati nello studio, i cui risultati sono illustrati nelle Dashboard 5.1 e 5.2 e sintetizzati nella Figura 5.3. Nello specifico:

- **Keyphrase più frequenti (Dashboard 5.1):** l'analisi delle keyphrase più frequenti evidenzia differenze sostanziali tra i due approcci considerati.
 - **en_core_web_sm**: le keyphrase più ricorrenti hanno contenuto semantico limitato rispetto agli obiettivi dell'analisi, infatti:
 - * la metà dei termini chiave, ovvero (**time**, **day**, **week**, **month**, **year**), sono riferimenti temporali generici, scarsamente informativi in termini tematici.
 - * la frequente occorrenza del termine **dell** riflette l'oggetto di studio del dataset, risultando pertanto ridondante e poco discriminante ai fini dell'estrazione di contenuti rilevanti.

L'unico elemento potenzialmente informativo è rappresentato dal termine *issue*, che suggerisce la presenza di numerose problematiche legate al prodotto o al servizio.

- **en_core_web_trf**: nonostante la frequente occorrenza dei termini *dell* e *laptop* che riflettono l'oggetto di studio del dataset, le espressioni estratte dal modello Transformer sono più contestualizzate e orientate all'esperienza dell'utente, includendo riferimenti espliciti a dimensioni relazionali (*customer service*, *your team*), contestuali (*my issue*), transazionali (*my order*, *my money*) e valutative (*awesome you guys*).

Per completezza, la specificità e la granularità delle keyphrase estratte dai modelli in analisi può essere osservata nelle wordcloud.

- **Relazione tra keyphrase e proprietà dei tweet (Dashboard 5.2)**: la dashboard presenta un confronto tra i modelli *spaCy* rispetto a due dimensioni analitiche:

- **Relazione tra lunghezza del tweet e numero di keyphrase estratte**: entrambi i modelli mostrano una tendenza positiva: all'aumentare della lunghezza del tweet cresce, in generale, il numero di keyphrase individuate. Tuttavia, la dispersione dei punti suggerisce che tale relazione non sia strettamente lineare, ma piuttosto caratterizzata da una certa variabilità, soprattutto per tweet di lunghezza intermedia. Questo indica che la lunghezza testuale costituisce una condizione favorevole, ma non sufficiente, per l'estrazione di un numero maggiore di unità informative.
- **Distribuzione del numero di keyphrase per tweet**: in questo contesto emergono differenze più marcate tra i modelli. L'approccio basato su **en_core_web_sm** presenta una concentrazione elevata di tweet con il numero massimo consentito di keyphrase (5), suggerendo una tendenza alla saturazione dell'estrazione. Al contrario, il modello **en_core_web_trf** mostra una distribuzione più bilanciata, con una maggiore frequenza di tweet contenenti 1 o 2 keyphrase e una progressiva diminuzione all'aumentare del numero estratto.

Nel complesso, i risultati suggeriscono che il modello transformer adotta un comportamento più selettivo e meno meccanicamente dipendente dal limite massimo di estrazione, mentre il modello small tende a raggiungere più frequentemente il numero massimo disponibile, indicando una minore capacità discriminativa.

- **Keyphrase overlap (Figura 5.3)**: per quantificare il grado di concordanza delle keyphrase estratte dai due modelli *spaCy* è stata utilizzata la *Jaccard similarity*, una misura di similarità adatta ai domini discreti definita come il rapporto tra la cardinalità dell'intersezione e quella dell'unione di due insiemi. Formalmente,

dati due insiemi A e B , la *Jaccard similarity* è calcolata come:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

In particolare, tale misura assume valori compresi tra 0 (nessuna sovrapposizione) e 1 (identità perfetta), e può essere ulteriormente discretizzata in tre intervalli significativi:

- buona sovrapposizione: $J(sm, trf) > 0.6$;
- parziale sovrapposizione: $0.3 \leq J(sm, trf) \leq 0.6$;
- marcata eterogeneità delle keyphrase estratte: $J(sm, trf) < 0.3$.

In questo contesto, l'analisi è stata condotta su due livelli distinti.

- **Tweet-level *Jaccard similarity*:** la *Jaccard similarity* si calcola per ciascun tweet confrontando gli insiemi di keyphrase estratti dal modello `en_core_web_sm` e dal modello `en_core_web_trf`. Il valore medio ottenuto (`Jaccard medio = 0.167`) indica una sovrapposizione relativamente bassa tra i due modelli a livello di singolo tweet. Ciò suggerisce che, pur operando sullo stesso testo, i modelli tendono a selezionare in larga parte keyphrase differenti, evidenziando strategie di estrazione divergenti.
- **Corpus-level *Jaccard similarity*:** a livello dell'intero dataset, la *Jaccard similarity* è calcolata tra gli insiemi globali di tutte le keyphrase uniche estratte dai due modelli. In questo caso il valore risulta leggermente superiore (`Jaccard globale = 0.185`), con 10.775 keyphrase condivise su un totale unificato di oltre 58.000. Questo dato indica che, pur mantenendo una forte componente di diversità lessicale, esiste un nucleo di keyphrase comuni che i due modelli identificano sistematicamente.

In conclusione, il confronto tra livello micro e macro suggerisce che la divergenza tra i modelli è più marcata a livello di singolo tweet, mentre a livello aggregato emerge una maggiore convergenza strutturale nel vocabolario estratto. In altri termini, i modelli tendono a selezionare keyphrase diverse nei singoli contesti, ma condividono in minima parte lo spazio semantico complessivo del corpus.

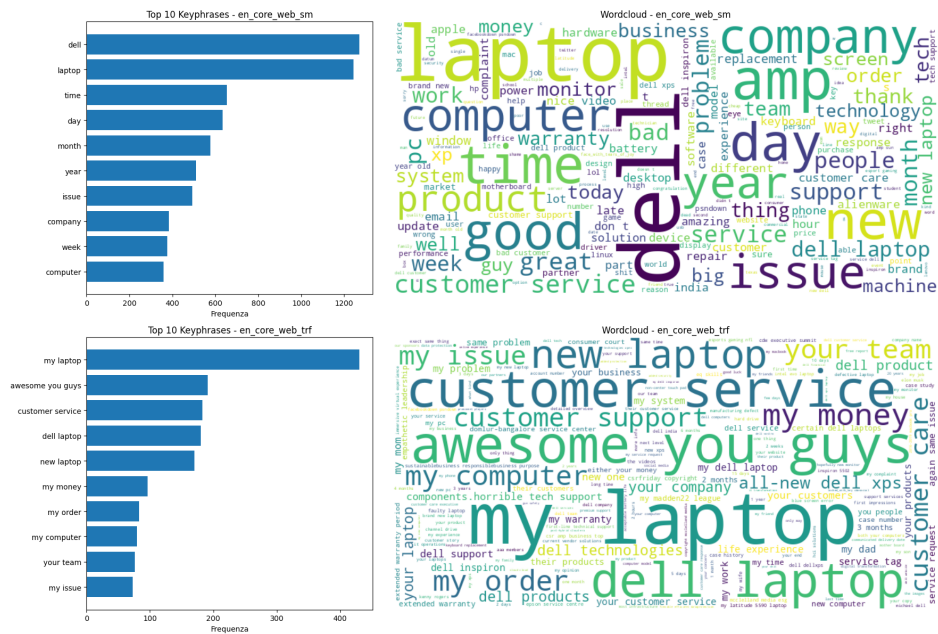


Figura 5.1: *Keyphrase più frequenti*

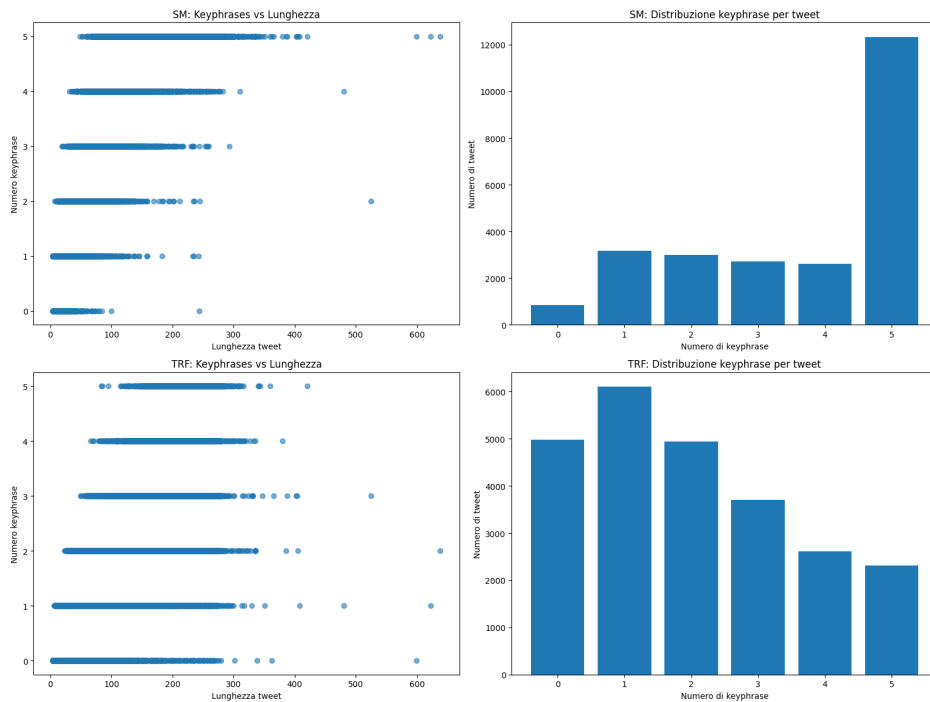


Figura 5.2: *Relazione tra keyphrase e proprietà dei tweet*

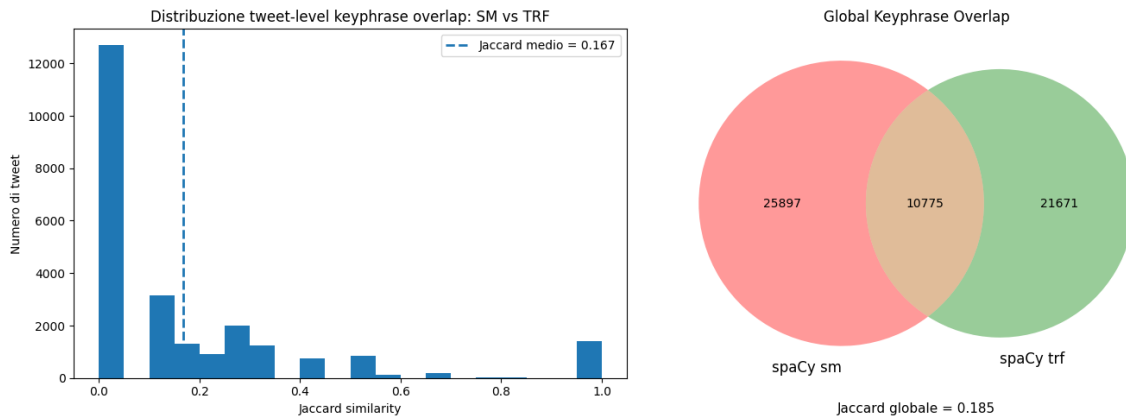


Figura 5.3: *Keyphrase overlap*

5.2 *Named Entity Recognition (NER)*

In questo capitolo viene descritto il processo di *Named Entity Recognition* applicato al corpus di tweet oggetto di analisi. Il task è preceduto da una fase di preprocessing finalizzata alla normalizzazione dei testi. Successivamente, l'identificazione e classificazione delle entità nominate viene effettuata mediante la pipeline di *spaCy*, sfruttando modelli pre-addestrati per la lingua inglese. Infine, vengono presentati e discussi i risultati sperimentali ottenuti, con particolare riferimento alla distribuzione delle etichette riconosciute e alla struttura informativa dei testi analizzati.

5.2.1 Pre-processing

Nell'ambito della *Named Entity Recognition (NER)*, la fase di pre-processing del testo è essenziale in quanto incide direttamente sulla qualità del riconoscimento delle entità. Nello specifico, il pre-processing persegue i seguenti obiettivi:

- Rimozione degli elementi non linguistici privi di contenuto informativo rilevante, come:
 - rimozione degli URL (`http`, `https`, `www`);
 - conversione delle emoji in testo descrittivo mediante la libreria `emoji` e il metodo `emoji.demojize(...)`;
 - rimozione del simbolo `#`, mantenendo tuttavia il testo dell'hashtag;
 - rimozione dei bullet point caratteristici degli elenchi puntati nei tweet;
 - rimozione dei caratteri speciali;
 - normalizzazione degli spazi multipli.

- Normalizzazione di alcune varianti testuali tipiche del linguaggio dei social media come, ad esempio, `can't` → `can not`, `ty` → `thank you`, `u` → `you`.

In particolare, a differenza dei task orientati alla riduzione della variabilità lessicale (come la *Key-Phrase Extraction*), nella *NER* i modelli sfruttano informazioni ortografiche e contestuali per l'identificazione delle entità, dunque è fondamentale preservare quanto più possibile le caratteristiche superficiali del testo, ovvero è necessario:

- Non convertire il testo lower-case: la capitalizzazione costituisce un segnale informativo rilevante per il riconoscimento di entità quali persone, organizzazioni o luoghi.
- Non rimuovere le menzioni (@): le menzioni degli utenti vengono preservate nella loro forma originale dal momento che possono corrispondere a riferimenti a persone, organizzazioni o eventi.

In conclusione, la pipeline di pre-processing implementata consente di ridurre il rumore tipico dei contenuti social preservando, al contempo, le informazioni necessarie per l'esecuzione del task *NER*.

5.2.2 *NER* mediante *spaCy*

La libreria *spaCy* implementa nativamente la *Named Entity Recognition* come componente della propria pipeline di elaborazione linguistica. In particolare, il task *NER* consiste nell'individuare e classificare automaticamente porzioni di testo (span) che fanno riferimento a entità del mondo reale, quali persone, organizzazioni, luoghi o altre categorie semanticamente definite.

Dal punto di vista architetturale, all'interno di *spaCy* la *NER* è realizzata tramite il componente denominato **EntityRecognizer**, responsabile dell'identificazione dei confini delle entità (entity boundary detection) e dell'assegnazione di entity-labels a specifiche sequenze di token. Questo componente opera a livello di documento (**Doc**) e produce come output una lista di entità accessibile tramite l'attributo `doc.ents`, dove ciascuna entità è rappresentata come uno span testuale accompagnato dalla relativa etichetta.

Dal momento che il dataset oggetto di studio, *Sentiment & Emotions Labelled Tweets*, è composto da tweet in lingua inglese, nel presente lavoro vengono impiegati i modelli pre-addestrati di *spaCy* `en_core_web_sm` e `en_core_web_trf`. In entrambi i casi, il riconoscimento delle entità è affidato al componente **EntityRecognizer**, che assegna etichette alle sequenze testuali sulla base di un modello supervisionato addestrato su corpora annotati.

In particolare, i modelli inglesi di *spaCy* adottano il seguente insieme predefinito di etichette che consentono una categorizzazione general-purpose delle principali tipologie di entità presenti nei testi:

- PERSON: nomi di persone reali o fittizie;
- ORG: organizzazioni, aziende, istituzioni;
- GPE (Geo-Political Entity): entità geopolitiche come paesi, città o stati;
- LOC: luoghi geografici non geopolitici (es. catene montuose, mari);
- FAC: strutture fisiche come edifici, aeroporti, ponti;
- NORP: nazionalità, gruppi religiosi o politici;
- PRODUCT: oggetti commerciali, veicoli, prodotti tecnologici;
- WORK_OF_ART: titoli di opere creative (libri, film, canzoni);
- EVENT: eventi storici, culturali o sportivi;
- LAW: documenti legislativi o atti giuridici;
- LANGUAGE: nomi di lingue;
- DATE: date assolute o relative;
- TIME: espressioni temporali inferiori al giorno;
- MONEY: valori monetari;
- PERCENT: percentuali;
- QUANTITY: quantità misurabili (peso, distanza, volume);
- ORDINAL: numeri ordinali (es. "first", "second");
- CARDINAL: numeri che non rientrano in altre categorie specifiche.

In conclusione, l'adozione congiunta di `en_core_web_sm` e `en_core_web_trf` consente di mettere a confronto due configurazioni architetturelle differenti: da un lato un modello più leggero e computazionalmente efficiente, dall'altro un modello basato su rappresentazioni contestuali profonde di tipo transformer.

5.2.3 Pipeline *NER*

Dal momento che la *NER* è parte integrante della pipeline *spaCy*, il seguente flusso di lavoro descrive il metodo congiunto di assegnazione delle label da parte dei modelli `en_core_web_sm` e `en_core_web_trf`.

- **Step 1:** si carica il modello pre-addestrato mediante le istruzioni
`nlp = spacy.load("en_core_web_sm"),`
`nlp = spacy.load("en_core_web_trf").` In questo modo, `nlp` diventa un oggetto pipeline che include tokenizer, tagger, parser e *NER*.
- **Step 2:** si applica la pipeline *spaCy* ai tweet mediante istruzione
`doc = nlp(tweet).`
- **Step 3:** si esegue mapping tra le entità estratte da ogni tweet e label standard definite in *spaCy*.

5.2.4 Risultati sperimentali *NER*

Nella sezione seguente si analizzano i risultati sperimentali ottenuti dall'applicazione delle tecniche di *Named Entity Recognition* al dataset di tweet oggetto di studio.

In particolare, la Tabella 5.3 include dieci tweet selezionati come benchmark, sui quali è stata condotta l'analisi comparativa. Tali esempi rappresentano un campione significativo del dataset, includendo menzioni di brand, persone, prodotti e riferimenti temporali tipici del linguaggio social. La Tabella 5.4 presenta invece i risultati prodotti dai due modelli impiegati, mettendo a confronto le entità riconosciute e le relative etichette assegnate. Il confronto consente di evidenziare differenze in termini di coerenza semantica tra il modello tradizionale basato su architettura leggera e quello basato su transformer.

Testi dei tweet di riferimento
@Logitech @apple @Google @Microsoft @Dell @Lenovo WhatIf QWERTY were modified for programmers so things like brackets parens quotes operators etc. moved to the middle near G-H splitting left right brackets to separate hands and relieving the less dextrous pinky finger
@MK habit addict @official stier @MortalKombat @newzealand She is getting a new @Dell laptop when the one she has one only 2-3 years old. More than triple the price - though much higher utility.
As @CRN celebrates its 40th anniversary Bob Faletra and @stevenjburke spoke with me about the milestones companies and personalities that helped build the channel. CRN40 podcast internationalpodcastday @Cisco @Microsoft @HPE @hp @Dell @intel
@dell your customer service is horrible especially agent syedfaisal who has made this experience of purchasing a new computer downright awful and I will reconsider ever buying a Dell in the future @DellTech
@zacokalo @Dell @DellCares @Dell give the man what he paid for
The screenshot is acting up from the website of @Google on my @Dell Latitude 5590 Laptop it keeps frozen the last time all the images are cut and all the videos are blind on @YouTube it is not working it is dead DellLatitude Laptop Google screenshot actingup frozen RIP
@emijuju @Alienware @Dell @IntelGaming alien shaped omurice
COOKING STREAM DAY Ty to @Alienware for sponsoring this stream I will be making a bunch of Japanese Alien themed foods hehe Come check it out alienware-partner intelgaming @Dell @IntelGaming
@emijuju @Alienware @Dell @intel Beautiful smiling face with heart-eyes red heart smiling cat with heart-eyes
What is your biggest data management challenge Cloud complexity Lengthy tech refresh cycles Capital budget constraints Solve your challenges with as-a-Storage. Get simplicity agility amp control with @Dell APEX.

Tabella 5.3: *NER: tweet di riferimento per il confronto tra modelli spaCy.*

Confronto tra modelli <i>spaCy</i>	
en_core_web_sm	en_core_web_trf
@Logitech → NORP @Google @Microsoft → ORG @Lenovo WhatIf → ORG	@Microsoft → ORG @Dell → ORG @Lenovo → ORG
one → CARDINAL	@Dell → ORG only 2-3 years old → DATE
@CRN → ORG 40th → ORDINAL Bob Faletra → PERSON @stevenjburke → ORG @Cisco @Microsoft @HPE → PERSON	@CRN → ORG 40th anniversary → DATE Bob Faletra → PERSON CRN40 → ORG @Cisco → ORG @intel → ORG
@DellTech → ORG	syedfaisal → PERSON) Dell → ORG @DellTech → ORG
\	@DellCares → ORG @Dell → ORG
@Google → ORG Latitude 5590 Laptop → FAC DellLatitude Laptop Google → ORG actingup frozen → PERSON RIP → ORG	@Google → ORG @Dell → ORG Latitude 5590 Laptop → PRODUCT @YouTube → ORG DellLatitude → ORG Google → ORG
@Alienware → ORG	\
@Alienware → ORG Japanese → NORP	Ty → PERSON @Alienware → ORG Japanese → NORP intelgaming → ORG @Dell → ORG @IntelGaming → ORG
@Alienware → ORG	\
Cloud complexity Lengthy → PERSON Solve → ORG APEX → EVENT	@Dell APEX → ORG

Tabella 5.4: Confronto entità ed etichette estratte con *spaCy* *en_core_web_sm* e *spaCy en_core_web_trf*.

In particolare, la Tabella 5.4 evidenzia che:

- **en_core_web_sm:** il modello mostra le seguenti criticità nella fase di riconoscimento e classificazione delle entità.
 - **Errata segmentazione:** si osservano casi di errata segmentazione, nei quali più menzioni distinte vengono aggregate in un'unica entità (ad esempio @Google @Microsoft → ORG, @Cisco @Microsoft @HPE → PERSON), compromettendo la corretta individuazione dei confini entitativi.
 - **Misclassificazione semantica:** il modello tende ad assegnare etichette non coerenti rispetto al contenuto testuale (ad esempio Cloud complexity Lengthy → PERSON, actingup frozen → PERSON).
 - **Fusione tra testo e hashtag:** si rilevano casi in cui testo e hashtag vengono fusi impropriamente in un'unica entità (ad esempio @Lenovo WhatIf → ORG).

Complessivamente, tali evidenze suggeriscono una minore accuratezza del modello, presenza di rumore, fenomeni di over-aggregation e attribuzioni etichettali non corrette.

- **Modello en_core_web_trf:** il modello presenta un comportamento più stabile e coerente.
 - Riconosce correttamente menzioni organizzative distinte (ad esempio @Microsoft → ORG, @Dell → ORG, @Lenovo → ORG), dimostrando una migliore capacità di segmentazione.
 - Si osserva una classificazione semantica più appropriata in presenza di espressioni temporali e riferimenti a prodotti (ad esempio 40th anniversary → DATE, Latitude 5590 Laptop → PRODUCT).
 - Identifica correttamente entità di tipo PERSON anche in assenza del simbolo @ (ad esempio syedfaisal → PERSON).
 - Mostra una gestione più accurata di contesti promozionali (ad esempio @Dell APEX → ORG).

In sintesi, il modello **en_core_web_trf** evidenzia una migliore capacità di segmentazione e una maggiore coerenza nell'assegnazione delle etichette rispetto alla versione **en_core_web_sm**, risultando più adatto all'analisi di testi brevi e rumorosi come i tweet. Permangono, tuttavia, alcune criticità.

- **Casi di falso positivo:** si osservano sporadici casi di falso positivo (ad esempio Ty → PERSON), che evidenziano come anche il modello basato su architettura transformer non sia esente da errori di classificazione. Una possibile motivazione a questo fenomeno è dovuta alle scelte di pre-processing

adottate: non essendo stata applicata una normalizzazione sistematica in lower-case, alcune espressioni brevi o contratte mantengono una forma ortografica che può essere interpretata come nome proprio. Infatti, in assenza di lower-casing, la gestione esaustiva delle varianti maiuscole/minuscole risulta computazionalmente onerosa, poiché una parola di lunghezza N può generare fino a 2^N combinazioni di casing (ad esempio, `Ty` produce $2^2 = 4$ varianti: `ty`, `TY`, `Ty`, `tY`). Dunque, tale variabilità può incidere negativamente sulla coerenza della classificazione.

- **Mancato riconoscimento di entità:** si rilevano casi di mancato riconoscimento di entità. Ad esempio, in presenza di più menzioni organizzative all'interno dello stesso tweet (quali `@Logitech`, `@apple`, `@Google`, `@Microsoft`, `@Dell`, `@Lenovo`), il modello restituisce solo un sottoinsieme delle entità effettivamente presenti (ad esempio `@Microsoft` $\rightarrow$ `ORG`, `@Dell` $\rightarrow$ `ORG`, `@Lenovo` $\rightarrow$ `ORG`). Questo fenomeno suggerisce possibili limiti nella copertura lessicale, nella sensibilità al contesto o nella gestione delle menzioni multiple in sequenza.

In relazione alle analisi svolte sui risultati riportati in Tabella 5.4, è opportuno precisare che le considerazioni relative ai punti di forza e alle criticità dei modelli `en_core_web_sm` e `en_core_web_trf` hanno natura esclusivamente qualitativa. Il dataset impiegato, infatti, non dispone di annotazioni *NER* di riferimento che consentano un confronto sistematico tra entità predette e entità corrette.

In assenza di etichette ground truth, non è pertanto possibile calcolare metriche quantitative standard quali *precision*, *recall* o *F1-score*, né effettuare una valutazione oggettiva delle prestazioni. Le osservazioni riportate derivano quindi da un'analisi manuale dei casi esemplificativi selezionati e devono essere interpretate come evidenze esplorative, utili a evidenziare tendenze comportamentali dei modelli piuttosto che a fornire una misurazione formale della loro accuratezza.

Sebbene l'analisi condotta presenti i limiti tipici di un'applicazione di *Named Entity Recognition* in assenza di annotazioni di riferimento, il confronto qualitativo tra i modelli evidenzia prestazioni mediamente più robuste da parte di `en_core_web_trf`.

Alla luce di tale evidenza, si è ritenuto opportuno approfondire in modo descrittivo il comportamento del modello `en_core_web_trf` attraverso un'analisi quantitativa esplorativa della distribuzione delle entità riconosciute. In particolare, i risultati dell'analisi sono mostrati nella Dashboard 5.4.

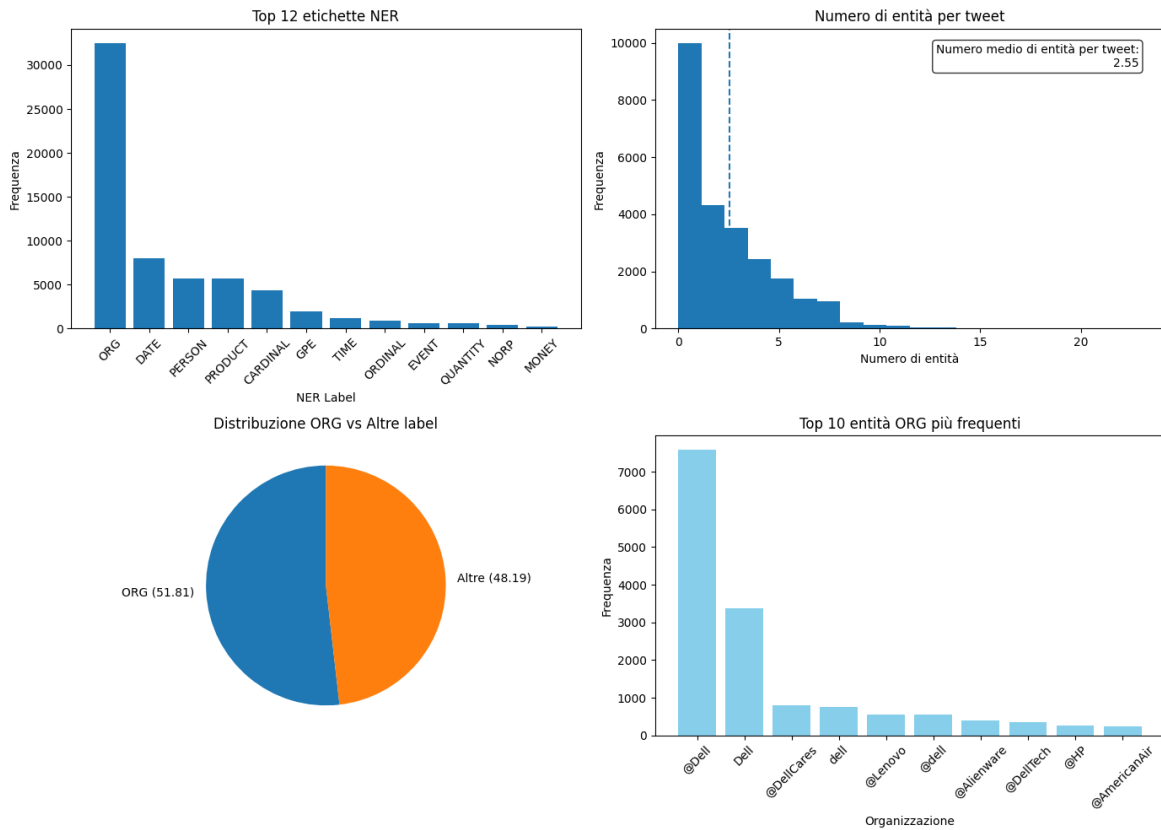


Figura 5.4: *Analisi della distribuzione delle NER labels*

Di seguito, si discutono i principali aspetti emersi dalla Dashboard 5.4.

- **Top 12 etichette *NER*:** il grafico a barre mostra la distribuzione delle etichette *NER*, utile a comprendere la prevalenza relativa delle diverse categorie semantiche. Sull'asse delle ascisse sono riportate le prime dodici label, mentre sull'asse delle ordinate è indicata la frequenza di occorrenza. La distribuzione evidenzia come la label **ORG** sia nettamente predominante con circa 33.000 occorrenze, seguita da **DATE** (8.000), **PERSON** e **PRODUCT** (circa 5.500 ciascuna).
- **Numero di entità per tweet:** Il grafico a barre rappresenta la distribuzione del numero di entità per singolo tweet, consentendo di valutare la densità informativa media dei testi. In particolare, la distribuzione assume un andamento monotono decrescente caratterizzato da un picco di tweet contenenti un'unica entità e una media per tweet pari a 2,55.
- **Distribuzione **ORG** vs altre label:** Il pie chart mostra la ripartizione tra la label **ORG** e tutte le altre categorie di entità. In particolare, **ORG** rappresenta il 51,81% del totale delle entità, mentre le altre label costituiscono il 48,19%, confermando il peso dominante delle organizzazioni nel dataset.

- **Top 10 entità ORG più frequenti:** Il grafico a barre mostra le dieci entità di tipo ORG con maggiore frequenza di occorrenza. La distribuzione evidenzia una forte concentrazione delle menzioni su Dell e sulle sue diverse varianti ortografiche e social (ad esempio @Dell, Dell, @DellCares, dell, @dell, DellTech). Tale predominanza risulta coerente con la natura del dataset, incentrato sul brand in oggetto, e contribuisce in modo determinante al peso complessivo della label ORG. Tuttavia, proprio questa concentrazione introduce uno sbilanciamento nella distribuzione delle entità: ai fini di un’analisi più equilibrata della struttura semantica del corpus, la Dashboard 5.5 mostra la una variante della Dashboard 5.4 caratterizzata dall’esclusione della menzione @Dell e delle sue varianti, così da ridurre l’effetto di sovra-rappresentazione e osservare più chiaramente la distribuzione delle restanti organizzazioni.

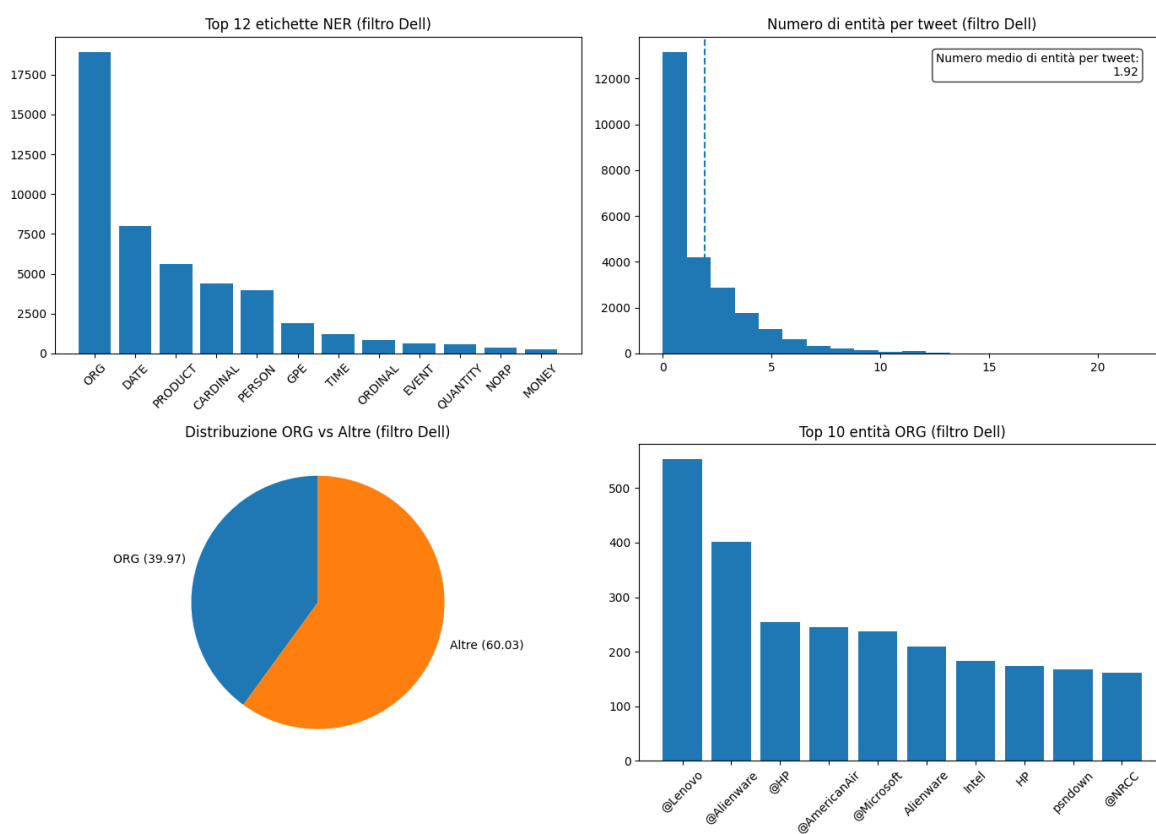


Figura 5.5: Analisi della distribuzione delle NER labels con filtro sulla keyword Dell